

HDOC - Day 18 Activity

C Programming Assignment

Name-Arushí Gusain

Sap id-590033489

Batch 15

Date- 11/09/2026

Problem

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, *with void return type and with a simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether it is a Strong number.
- Check whether it is an Armstrong number.
- Check whether it is a Harshad number.
- Check whether it is a Duck number.

Concepts Used

Strong number: A number equal to the sum of factorials of its digits.

Example: $145 = 1! + 4! + 5! = 145$.

Armstrong number: A number equal to the sum of each digit raised to the power of the total number of digits.

Example: $153 = 1^3 + 5^3 + 3^3 = 153$.

Harshad number: A positive integer divisible by the sum of its digits.

Example: 18; digit sum = 9 and $18 \% 9 = 0$.

Duck number: A positive integer containing at least one zero digit (leading zeroes are not considered). Example: 1023.

Algorithm

Main program

1. Start.
2. Display four menu options: Strong, Armstrong, Harshad and Duck number.
3. Read the user choice.
4. Use switch(choice) to call exactly one corresponding void function.
5. For an invalid choice, display an error message.
6. Stop.

Strong Number Function

1. Read a positive integer n.
2. If $n \leq 0$, display invalid input and return.
3. Set temp = n and sum = 0.
4. Extract each digit using temp % 10.
5. Find factorial of the digit and add it to sum.
6. Remove the last digit using temp /= 10.
7. If sum == n, display Strong number; otherwise display Not a Strong number.

Armstrong Number Function

1. Read a positive integer n.
2. Count the number of digits in n.
3. For every digit, compute digit raised to the number of digits using repeated multiplication.
4. Add these powers to sum.
5. If sum == n, display Armstrong number; otherwise display Not an Armstrong number.

Harshad Number Function

1. Read a positive integer n.
2. Find the sum of all digits.
3. If $n \% \text{sum} == 0$, display Harshad number; otherwise display Not a Harshad number.

Duck Number Function

1. Read a positive integer n.
2. Examine each digit.

3. If any digit is 0, set a flag and stop checking.
4. Display Duck number if the flag is set; otherwise display Not a Duck number.

Test Case Table

Case	Choice	Input	Expected Output	Reason	Result
1	1	145	Strong number	$1! + 4! + 5! = 145$	Pass
2	1	123	Not a Strong number	$1! + 2! + 3! = 9$	Pass
3	2	153	Armstrong number	$1^3 + 5^3 + 3^3 = 153$	Pass
4	2	125	Not an Armstrong number	$1 + 8 + 125 = 134$	Pass
5	3	18	Harshad number	Digit sum 9; $18 \% 9 = 0$	Pass
6	3	19	Not a Harshad number	Digit sum 10; $19 \% 10 \neq 0$	Pass
7	4	1023	Duck number	Contains digit 0	Pass
8	4	1234	Not a Duck number	Contains no zero	Pass
9	5	-	Invalid choice	Choice is outside 1-4	Pass

C Program

```
#include <stdio.h>
void strongNumber(void);
void armstrongNumber(void);
void harshadNumber(void);
void duckNumber(void);
int main(void)
{
    int choice;
    printf("DIGIT OPERATIONS MENU\n");
    printf("1. Check Strong number\n");
    printf("2. Check Armstrong number\n");
    printf("3. Check Harshad number\n");
    printf("4. Check Duck number\n");
    printf("Enter your choice: ");
```

```

scanf("%d", &choice);
switch (choice) {
case 1: strongNumber(); break;
case 2: armstrongNumber(); break;
case 3: harshadNumber(); break;
case 4: duckNumber(); break;
default: printf("Invalid choice.\n");
}
return 0;
}
void strongNumber(void)
{
long long n, temp, sum = 0;
int digit, i;
long long fact;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) {
printf("Invalid input. Enter a positive integer.\n");
return;
}
temp = n;
while (temp > 0) {
digit = temp % 10;
fact = 1;
for (i = 1; i <= digit; i++)
fact *= i;
sum += fact;
temp /= 10;
}

if (sum == n)
printf("%lld is a Strong number.\n", n);
else
printf("%lld is not a Strong number.\n", n);

```

```

}
void armstrongNumber(void)
{
    long long n, temp, sum = 0, power;
    int digit, digits = 0, i;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    if (n <= 0) {
        printf("Invalid input. Enter a positive integer.\n");
        return;
    }
    temp = n;
    while (temp > 0) {
        digits++;
        temp /= 10;
    }
    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        power = 1;
        for (i = 0; i < digits; i++)
            power *= digit;
        sum += power;
        temp /= 10;
    }
    if (sum == n)
        printf("%lld is an Armstrong number.\n", n);
    else
        printf("%lld is not an Armstrong number.\n", n);
}
void harshadNumber(void)
{
    long long n, temp, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);

```

```

if (n <= 0) {
printf("Invalid input. Enter a positive integer.\n");
return;
}
temp = n;
while (temp > 0) {
sum += temp % 10;
temp /= 10;
}
if (n % sum == 0)
printf("%lld is a Harshad number.\n", n);
else
printf("%lld is not a Harshad number.\n", n);
}

void duckNumber(void)
{
long long n, temp;
int isDuck = 0;
printf("Enter a positive integer: ");
scanf("%lld", &n);
if (n <= 0) {
printf("Invalid input. Enter a positive integer.\n");
return;
}
temp = n;
while (temp > 0) {
if (temp % 10 == 0) {
isDuck = 1;
break;
}
temp /= 10;
}
if (isDuck)
printf("%lld is a Duck number.\n", n);
else

```

```
printf("%lld is not a Duck number.\n", n);
}
```

Compilation , Execution & Testing

```

Session Actions Edit View Help
(kali㉿kali)-[~]: number is palindrome
$ vi duck.c
Enter a positive integer: 11052
(kali㉿kali)-[~]: number = 10235
$ cc duck.c

(kali㉿kali)-[~]:
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 145902
145 is a Strong number.

(kali㉿kali)-[~]:
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong number.

(kali㉿kali)-[~]:
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.

```

```

Session Actions Edit View Help
(kali㉿kali)-[~]: number is palindrome
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2
Enter a positive integer: 125
125 is not an Armstrong number.

(kali㉿kali)-[~]: number is palindrome
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.

(kali㉿kali)-[~]: number is palindrome
$ ./a.out
DIGIT OPERATIONS MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 19
19 is not a Harshad number.

```